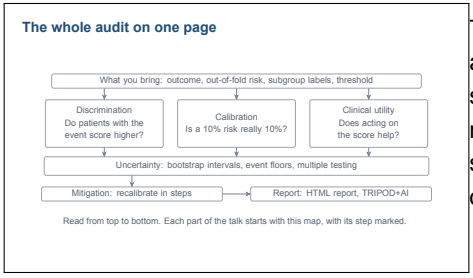


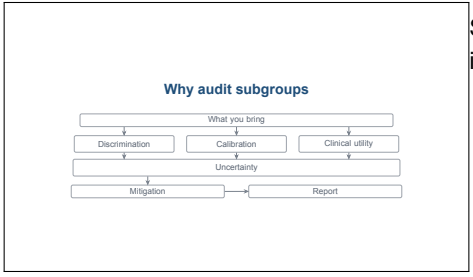
This talk shows how to check a clinical prediction model in each patient subgroup as well as in the whole cohort. The method is general. The worked example is our model that estimates the risk of infection after surgery, at the end of the operation. Its audit is described in a paper accepted at JAMIA in 2026. Every analysis in that paper is a call to the isitfair package, so the code on these slides is the code that produced the paper's numbers.

The talk has nine parts

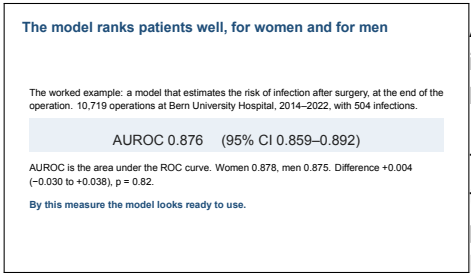
Part 1 shows why an audit is needed, with the worked example. Parts 2 to 6 are the three things we measure: discrimination, calibration and clinical utility. Part 7 is about uncertainty, which is where most mistakes happen. Part 8 is mitigation, and part 9 is the report. The parts follow the order in which the questions come up in a real audit.



This is the map of the audit. You bring four inputs. The audit measures three things. All three go through the same uncertainty rules. Only then do you decide on mitigation, and finally you write the report. The arrows show the order. I will come back to this map at the start of each part.



Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.



AUROC 0.876 means: take one patient who got an infection and one who did not. The model gives the higher score to the infected patient about 88 times in 100. Women and men get almost the same value. These are out-of-fold predictions, so the model did not train on the patients it scored. The model itself was published earlier by Blatter and colleagues in npj Digital Medicine, 2026. Ask the room: would you use this model on the ward? Most people say yes at this point. The next three slides show why that answer is too early.

The risks the model prints are too high

Compare the printed risk with what happened to those patients:

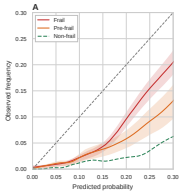
The model shows	5%	10%	20%	30%
Observed infection rate	0.9%	2.1%	8.3%	18.3%

Observed rates come from the smoothed calibration curve of all 10,719 operations. On average, the printed risk is 6.2 percentage points away from the observed rate.

Part of this error comes from the data. Before 2018, fewer infections were recorded: 4.70% of operations, against 8.28% from 2018 onward.

The model puts patients in nearly the right order, but the numbers it prints are too high. The error is largest at low printed risk: at 10%, only 2.1% of those patients had an infection. A clinician who trusts the number would monitor, delay discharge or give antibiotics to many patients who did not need it. The data limitation matters: before 2018 the hospital record rarely showed whether an infection started after surgery. Those infections left the cohort, so the early years have very few recorded infections. Part 5 shows how much this changes the calibration numbers. AUROC hardly changes.

The size of the error depends on physiological reserve



Physiological reserve: how well a patient can tolerate a complication. It is built from the hospital record.

When the model shows 20%, the observed rate is:

Frail	9.6%
Pre-frail	5.8%
Non-frail	2.2%

The non-frail subgroup has only 6 infections. Its curve is dashed and has no interval.

Physiological reserve is a three-level label built from routine data: ASA class, the Hospital Frailty Risk Score and a short frailty index. Part 2 gives the exact definition. The model overestimates risk in all three subgroups, and more so as reserve increases. The curves come from LOESS smoothing with 95% bootstrap bands. The non-frail value is shown, but we do not interpret it: six infections cannot support an interval. Part 7 explains this rule.

The usefulness of the score differed by physiological reserve

Net benefit at a 10% threshold, per 1,000 operations, 2014–2022:

Frail	+17.21 (+10.61 to +23.73)
Pre-frail	−10.59 (−15.77 to −5.37)

A negative value means that acting on the score did more harm than acting on no one.

The pre-frail result did not hold in later data: +3.48 (−5.80 to +13.55) in 2018–2022, and +14.72 (−3.49 to +34.00) in a separate test on 2021–2022.

Net benefit is explained in part 6. For now: it counts the infections the model finds and subtracts the false alarms, weighted by the threshold. In frail patients the model helps. In pre-frail patients, in the full cohort, acting on it was worse than doing nothing. We report this result together with its second half every time. In 2018–2022 the interval includes zero, and in the separate 2021–2022 test the value is positive. Part of the difference is arithmetic: pre-frail recorded prevalence was 2.75% in the full cohort and 5.27% in 2018–2022, and net benefit falls when there are fewer cases. So this is a finding about the source cohort. It is not a stable property of the model. AUROC could not show any of this, because it ignores the risk values.

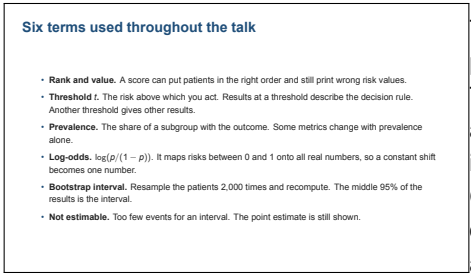
This tutorial follows the order of a real audit

1. What you must bring, and what makes your inputs unusable.
2. Three axes: what each measures, how to estimate it, how to read it.
3. Uncertainty: what an interval means, and when to withhold it.
4. Mitigation, in the order to try it.
5. A report you can give to a reviewer.

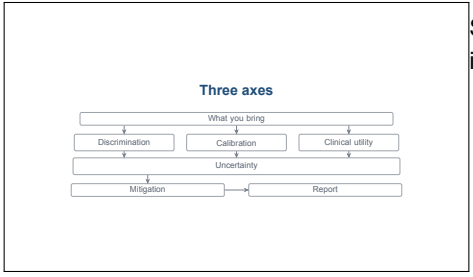
Slides with a formula also say it **in words**. **Common mistake** marks the error a slide helps you avoid.

All code on these slides runs on isitfair 0.4.

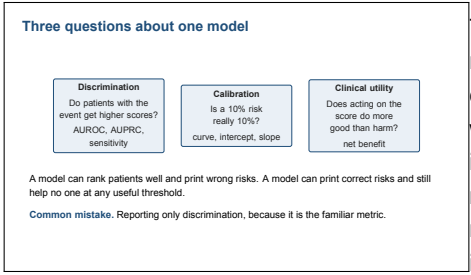
The method does not depend on the worked example. You can run the same steps on any binary prediction model, as long as you have its predicted probabilities for patients it did not train on. Each code block on these slides is run automatically against the package before the slides are built, so the code should work as shown.



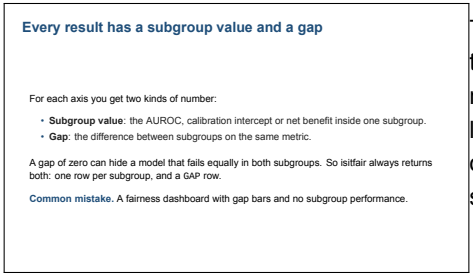
These six terms come from the general evaluation of prediction models. Everything later is built from them. The most important is the first one: a score has an order and a value, and they can be right or wrong independently. Discrimination checks the order. Calibration checks the value. An event means the outcome happened; in the worked example, an event is an infection.



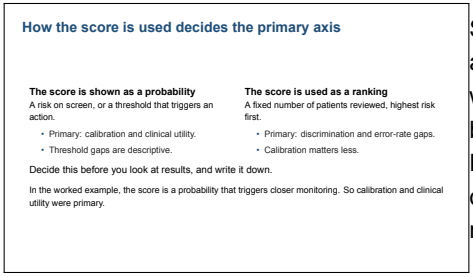
Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.



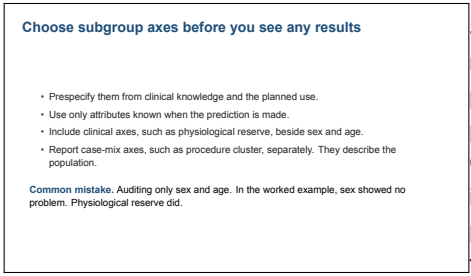
The three axes answer three different questions, and none of them follows from another. In the worked example the first axis looked excellent and the second was clearly wrong. The third axis turns calibration errors into consequences for patients. TRIPOD+AI, the reporting guideline for prediction models that use machine learning, asks for performance with uncertainty in relevant subgroups. That means all three axes.



Two subgroups can have the same sensitivity because the model is equally poor in both. The gap is zero, and nobody is served well. If you show only gaps, that result looks like success. This is why the package has no option to return gaps alone. Every table has the subgroup rows and the gap row together.



Subgroups can differ because their patients differ. Even a perfect model shows different subgroup performance when case mix or prevalence differ. So a difference between subgroups is not by itself evidence of a fault. Read every gap against case mix and prevalence, and do this the same way on every axis. Do not use case mix to explain only the gaps you do not like.



Searching the data for the worst subgroup is a different method, with different error control. An audit tests prespecified axes. An axis you only learn after the prediction cannot inform the decision you are auditing. Case-mix axes, like the type of operation, describe who is in the cohort. They are useful context, but they are not patient groups that the model owes equal performance to. In the worked example, recorded sex was linked to reserve and to the type of operation, so a sex-only audit would have looked reassuring and missed the problem.

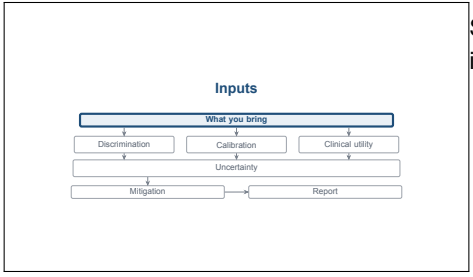
Physiological reserve in the worked example

Label	Any of these	Operations	Infections
Frail	ASA ≥ 4, HFRS > 15, mFI-4 ≥ 2	6,139	400
Pre-frail	ASA 3, HFRS 5–15, mFI-4 = 1	3,558	98
Non-frail	none of these	1,022	6

ASA: American Society of Anesthesiologists physical status. HFRS: Hospital Frailty Risk Score. mFI-4: a four-item modified frailty index.

The labels describe a proxy built from the hospital record. They are not diagnoses of frailty.

Pre-frail requires that no frail criterion is met. The mFI-4 items are diabetes, chronic obstructive pulmonary disease, heart failure and hypertension. Hypertension came from diagnosis codes without medication data, so it may be overcounted. The paper checks a version without the frailty index and the results are similar. Note the very small non-frail subgroup: 6 infections. It appears on almost every later slide, always without an interval.



Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.

Bring four inputs

1. `y_true`: the outcome, 0 or 1, one per patient.
2. `y_proba`: the predicted probability, for patients the model did not train on.
3. `sensitive_features`: a dict of subgroup labels, kept apart from the model's features.
4. `threshold`: one threshold t, fixed in advance.

isitfair never sees the model or its features. It evaluates predictions.

Any model works, in any language, if you can export its predicted probabilities. The subgroup labels are passed separately on purpose: they are for evaluation and do not need to be model inputs. The threshold is the risk at which a clinician would act. Fix it before you look at any result, because every threshold-based result depends on it.

Notation used in the formulas

p_i is the predicted risk of patient i, and $Y_i \in \{0, 1\}$ the outcome.

For subgroup g: I_g its patients, $n_g = |I_g|$ their number, n_g^+ its events, and $\pi_g^+ = n_g^+ / n_g$ its prevalence.

A patient is flagged when $p_i \geq t$. This gives the counts $TP_g(t)$, $FP_g(t)$, $FN_g(t)$ and $TN_g(t)$.

An event is an outcome that happened. In the worked example, an event is an infection.

TP, FP, FN and TN are true positives, false positives, false negatives and true negatives: the four cells of the two-by-two table at threshold t, counted inside subgroup g. Every threshold-based metric later is a ratio of these counts.

Use predictions from patients the model did not train on

A model is too optimistic on patients it trained on. This is worst in small subgroups.

- With cross-validation, use each patient's prediction from the fold that left the patient out.
- Resampling such as SMOTE (synthetic minority oversampling) belongs inside the training folds only.

In the worked example, the model was trained with SMOTE. Refitting without it moved the overall calibration intercept by +0.94 logits (0.91 to 0.96), in 2018–2022 data. The frail minus pre-frail difference grew by 0.17.

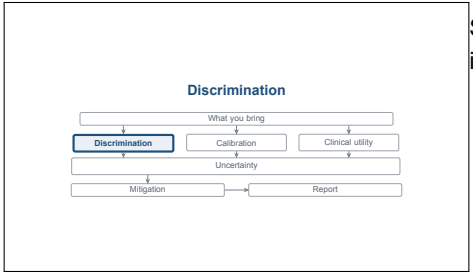
Overfitting is worst where data are sparse, which means the small subgroups an audit cares most about. A recalibration is also a fitted model, so it needs the same treatment; part 8 shows how. On SMOTE: creating synthetic cases shifts all predicted risks upward, as Van den Goorbergh and colleagues showed in JAMIA in 2022. In the worked example SMOTE explained a large part of the overall shift. But removing it made the difference between frail and pre-frail patients larger. So class rebalancing explains the average error and not the subgroup difference. This comparison was run in the 2018–2022 window, where the outcome is recorded.

Construct the audit

```
from isitfair import FairnessAudit

audit = FairnessAudit(
    y_true=y,              # 0/1 outcome
    y_proba=p_oof,         # out-of-fold predicted probability
    sensitive_features={
        "sex": sex,
        "age_group": age_group,
        "reserve": reserve,
        ("sex", "reserve"): None,   # intersection, labels built for you
    },
    threshold=0.10,        # fixed before you look at results
    n_bootstrap=2000,
    random_state=42,       # same seed and data give the same output
)
```

Nothing is computed yet. Each axis is computed when you call it, and then stored. A tuple key defines an intersection, here sex by reserve. Both parts of the tuple must also be given as single keys, because an intersection is defined relative to its parent subgroups. Otherwise the constructor raises an error. Missing values in any input also raise an error that names the input. With random_state fixed, the same data always give the same output, to the last digit.



Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.

AUROC measures how well the model ranks patients

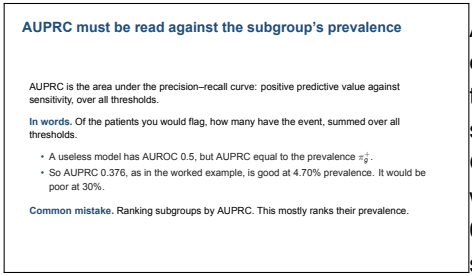
$$\mathrm{AUROC}_g = \Pr\left(p_i > p_j \mid i,j \in I_g,\ Y_i = 1,\ Y_j = 0\right)$$

In words. Take one patient with the event and one without, from the same subgroup. AUROC is the probability that the model gives the higher score to the patient with the event.

- It needs no threshold.
- It changes little when prevalence changes.

Common mistake. Comparing subgroup AUROCs as if all scores were on one scale. Each AUROC compares patients only inside its own subgroup.

AUROC is estimated with the Mann–Whitney rank statistic, with ties counted as one half. A useless model has AUROC 0.5. Because AUROC compares patients inside one subgroup, two subgroups can both have high AUROC while the model gives one of them systematically higher scores. AUROC cannot see that. Calibration can.



At low prevalence, AUROC is dominated by the many easy patients without the event. AUPRC is not, so the two can disagree about which subgroup is better served. Saito and Rehmsmeier showed this in PLoS One in 2015. In the worked example, pre-frail AUPRC was 0.190 at 2.75% prevalence and frail AUPRC was 0.428 at 6.5%. Compare each value with its own subgroup's prevalence.

A threshold turns the score into a decision rule

$$\mathrm{TPR}_g = \frac{TP_g}{TP_g + FN_g}; \quad \mathrm{FPR}_g = \frac{FP_g}{FP_g + TN_g}; \quad \mathrm{PPV}_g = \frac{TP_g}{TP_g + FP_g}; \quad \mathrm{NPV}_g = \frac{TN_g}{TN_g + FN_g}$$

TPR is sensitivity, FPR the false-positive rate, PPV and NPV the positive and negative predictive values.

In words. Sensitivity and FPR are shares of the patients with and without the event. PPV and NPV are shares of the flagged and unflagged patients. PPV and NPV change with prevalence even when the model does not change.

Common mistake. Calling a PPV difference bias when prevalences differ.

Specificity is one minus the false-positive rate. Take two subgroups at 1% and 10% prevalence and one common threshold. Their PPV must differ, even with an identical model, because of arithmetic alone. So a PPV gap between subgroups with different prevalence is expected. Sensitivity and FPR are more stable properties of the model at that threshold.

Run discrimination

```
disc = audit.discrimination()
disc[disc.subgroup != "GAP"][["attribute", "subgroup", "n_events",
                              "auroc", "auroc_ci_low", "auroc_ci_high"]]
```

One long table. For each attribute: one `Overall` row, one row per subgroup and one `GAP` row per gap statistic.

Keys and size	`attribute, subgroup, shrunk, n, n_events, prevalence`
Metrics	`auroc, auprc, sensitivity, specificity, ppv, npv` each with `_ci_low` and `_ci_high`
Gaps	`gap_metric, gap_value, gap_ci_low, gap_ci_high`
Rules	`n_replicates, n_estimable, proportion_nonestimable, not_estimable, event_floor, below_floor, not_estimable_final`

Every axis returns the same long format, so you can filter and join the tables in the same way. The last group of columns says whether a result may carry an interval. Part 7 explains those rules. The column shrunk marks intersection cells pulled toward their parent subgroup, explained in part 9.

Four parity gaps compare one rate between subgroups

For two subgroups g and g' at the same threshold t:

$\Delta_{\mathrm{EOpp}} = \mathrm{TPR}_g - \mathrm{TPR}_{g'}$	equal opportunity: sensitivity gap
$\Delta_{\mathrm{PE}} = \mathrm{FPR}_g - \mathrm{FPR}_{g'}$	predictive equality: false-alarm gap
$\Delta_{\mathrm{SP}} = \frac{TP_g + FP_g}{N_g} - \frac{TP_{g'} + FP_{g'}}{N_{g'}}$	statistical parity: flag-rate gap
$\Delta_{\mathrm{EO}} = \max\left(\lvert\Delta_{\mathrm{EOpp}}\rvert, \lvert\Delta_{\mathrm{PE}}\rvert\right)$	equalized odds

In words. One rate, subtracted between two groups. With more than two subgroups, the gap is the largest difference over all pairs.

Common mistake. Treating these as properties of the model. They describe the decision rule at one threshold.

In the table they are called equal_opportunity_difference, predictive_equality, statistical_parity and equalized_odds_gap. Change the threshold and all four change. So report them at the prespecified threshold, and treat them as descriptions of the decision rule. In the worked example they were descriptive, because the score is used as a probability.

Cohen's *h* shows whether a gap is large

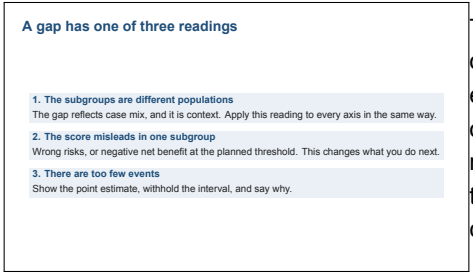
$$h(p_g, p_{gr}) = 2\left(\arcsin\sqrt{p_g} - \arcsin\sqrt{p_{gr}}\right)$$

In words. A difference between two proportions, rescaled so that a gap near 0 or 1 counts more than the same gap near 0.5.

- A 2-point sensitivity gap means more at 95% than at 50%.
- Usual reading: 0.2 small, 0.5 medium, 0.8 large.
- Leave *h* blank where it does not apply: equalized odds, calibration intercept and slope, net benefit.

Common mistake. Reading a significant gap as a large gap. In a large cohort almost every gap is significant.

The arcsine square root stabilises the variance of a proportion, which is why h treats gaps near the edges as larger. For composite gaps like equalized odds, and for metrics that are not proportions, there is no single matching effect size. A blank cell is better than a misleading number.



The first reading applies when, for example, types of operation differ in baseline infection risk. That is expected. The rule is consistency: you may not explain only the inconvenient gaps by case mix. The second reading is the finding that matters for deployment. The third is covered in part 7. Every gap you report should carry one of these three readings.

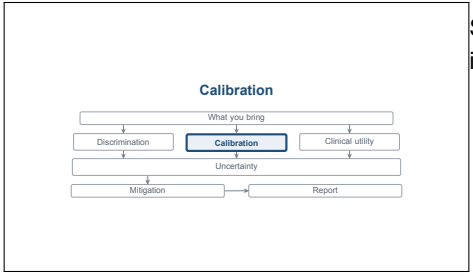
Age showed a threshold gap that sex did not

At $t = 0.10$:

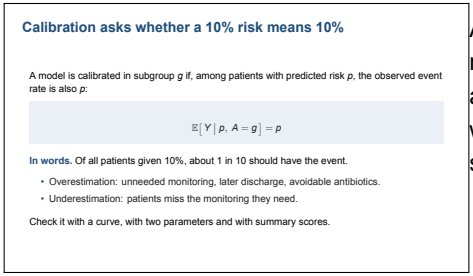
Age	Operations	Infections	AUROC	Sensitivity	Specificity
0–17	367	21	0.807	0.857	**0.462**
18–44	941	34	0.854	0.824	**0.682**
45–64	3,715	147	0.888	0.918	0.666
65–79	4,684	249	0.889	0.912	0.654
80 or older	1,012	53	0.805	0.849	0.577

Equalized-odds gap: 0.010 for sex, 0.220 for age. It comes from specificity: one threshold flags many more uninfected children. Age 0–17 has 21 infections, below the 25-event floor, so its row has no interval.

The marked rows produce the age gap: specificity 0.462 in children and 0.682 in young adults. A single global threshold flags many more children who do not get an infection. AUROC is also lower at both age extremes. The child subgroup is below the event floor, so we show its numbers without intervals and do not interpret them. The lesson is to audit every prespecified axis, and not to stop when the axis you were asked about looks fine.



Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.



A is the subgroup attribute. Calibration is the axis that matters when a number reaches a clinician or triggers an action. Ranking cannot replace it. Each of the three ways of checking it misses something the others show, so the package reports all three in every subgroup.

The calibration curve shows where the model is wrong

Plot the observed event rate against the predicted risk, smoothed by LOESS (local regression), with a 95% bootstrap band.

- The diagonal is perfect calibration. Below it, the model overestimates risk.
- Draw each subgroup's curve only over its own range of predictions.
- In a subgroup with few events, draw the curve dashed and without a band.

No single number shows whether the error is small everywhere or large in one part of the risk range. The curve shows it.

LOESS estimates, at each point on the risk scale, the share of nearby patients who had the event. How wide "nearby" is, the span, is a choice, just like the number of bins in the expected calibration error. Drawing a curve beyond the subgroup's observed predictions shows extrapolation as if it were data. A common mistake is to read a bend in a sparse subgroup as real structure. Check the band and the event count first.

The calibration intercept measures the average shift

With log-odds $l_i = \log(p_i/(1-p_i))$, fit one parameter, with the slope fixed at 1:

$$\text{logit}\,\Pr(Y_i = 1) = \alpha_g + l_i, \qquad i \in I_g$$

In words. One number for "every risk is off by the same amount".

- $\alpha_g = 0$: right on average.
- $\alpha_g < 0$: risks too high. $\alpha_g > 0$: risks too low.

In the worked example, α = −1.02 overall.

This is also called calibration-in-the-large. Fixing the slope at 1 is deliberate: it asks whether the whole risk scale is shifted, without letting a slope absorb part of the shift. In the worked example, minus 1.02 means the printed odds are about 2.8 times the observed odds on average.

AUROC held between data windows and the intercept moved

	2014–2022	2018–2022
Operations	10,719	6,051
Infections	504	501
Recorded prevalence	4.70%	8.28%
AUROC	0.876	0.855
Calibration intercept α	−1.02	−0.58

Before 2018, an infection needed a present-on-admission code to enter the data, and the code was rarely recorded. Report every result that depends on prevalence together with its data window.

This is the study's own example of why the intercept depends on prevalence. The present-on-admission code shows whether an infection was already there when the patient arrived. Without it, the timing of an infection cannot be established, so those cases left the cohort. They were not counted as uninfected. The early years therefore contribute only 3 infections from 4,668 operations. Ranking is stable across the two windows. The intercept, positive predictive value and net benefit are not. The audit keeps 2014–2022 as primary because that is the cohort the deployed model was developed on, and reports 2018–2022 next to it.

The calibration slope measures the spread of the risks

Now fit both terms:

$$\text{logit}\,\Pr(Y_i = 1) = \alpha_g^{(2)} + \beta_g\, k_i, \qquad i \in I_g$$

In words. Are the risks spread too wide or too narrow?

- $\beta_g = 1$: correct spread.
- $\beta_g < 1$: too spread out, typical of overfitting.
- $\beta_g > 1$: too compressed, typical after class rebalancing.

In the worked example, β = 1.57. The intercept $\alpha_g^{(2)}$ of this fit is not the calibration intercept. Do not report it as α.

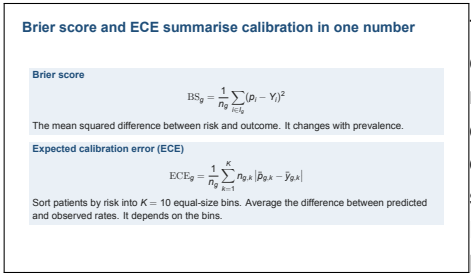
The slope describes the spread of the risks, and the intercept their average level. They answer different questions. When the slope is free, the intercept changes meaning: it becomes the shift at log-odds zero, which differs from the average shift. That is why the package fits the two models separately.

Read the intercept and slope together

Pattern	Meaning
$\alpha \approx 0$, $\beta \approx 1$	Calibrated.
$\alpha < 0$, $\beta \approx 1$	Every risk too high by one constant on the log-odds scale.
$\beta < 1$	Risks too extreme. They need a slope correction.
$\beta > 1$	Risks too compressed. They need a slope correction.

The worked example had both problems: $\alpha = -1.02$ and $\beta = 1.57$. One recalibration for all patients fixed both: α −0.007, β 0.93.

A pure shift is the easiest case: one common recalibration fixes it. The worked example had a large shift on top of compressed risks. Common recalibration was still the right first step, and it fixed both without using subgroup membership. Part 8 shows the recalibration. A common mistake is to start with subgroup-specific correction before removing the shift that affects everyone.



The Brier score rewards both good ranking and good calibration. It is a proper scoring rule: it is best when the model reports its true belief. But a subgroup with few events gets a low Brier score almost automatically, so do not compare it between subgroups. Use it inside one subgroup, for example before and after recalibration. For ECE, patients with tied risks are kept in the same bin.

ICI measures the same gap without bins

Integrated calibration index (ICI)

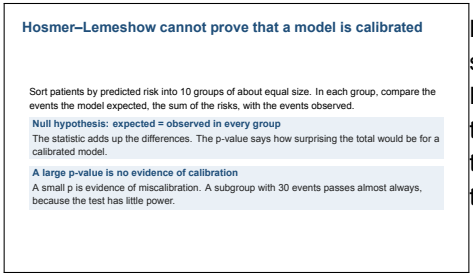
$$\mathrm{ICI}_g = \frac{1}{n_g} \sum_{i \in I_g} |f_g(p_i) - p_i|$$

The average distance of each patient's risk from the smoothed calibration curve f_g.

ECE and ICI usually agree. If they do not, the bins drive ECE: look at the curve before you report either.

In the worked example: ECE 0.062 and ICI 0.061.

ICI needs no bins. It depends only on the smoother. Both numbers are in units of risk: 0.062 means an average error of 6.2 percentage points. That is large for an outcome with 4.70% prevalence.



Never write that the Hosmer–Lemeshow test was not significant, so the model is calibrated in this subgroup. Report the test descriptively, next to ECE, ICI and the table of observed and expected events per group. The table shows where the misfit is. The statistic only shows that there is one.

Use g degrees of freedom for out-of-fold predictions

With O_k and E_k the observed and expected events in group k:

$$\chi^2_{\mathrm{HL}} = \sum_{k=1}^{g} \frac{(O_k - E_k)^2}{E_k\,(1 - E_k/n_k)}$$

In words. Each group's miss, squared, divided by how much it could miss by chance. Compare the total with a χ^2 distribution.

- $g - 2$ applies when the model was fitted on the data under test.
- Out-of-fold predictions fit nothing on that data. Use g.

Common mistake. Comparing H–L between subgroups of different size. It grows with n at the same lack of fit.

The g minus 2 convention spends two degrees of freedom on an intercept and slope estimated from the same data. Out-of-fold predictions estimate nothing on the data under test, so no degrees of freedom are spent. Using g minus 2 here makes the test find miscalibration too often. Software packages differ on this, so state the convention you used.

Run calibration

```
cal = audit.calibration()
# calibration_intercept, calibration_slope, brier,
# ece, ici, and GAP rows per attribute

hl = audit.hosmer_lemeshow()              # df = g by default
deciles = audit.hosmer_lemeshow(detail=True)
# per risk group: observed, expected, difference

curves = audit.calibration_curves()       # LOESS curve and band
```

Pass `df_mode="development"` only if the model was fitted on these data. Publish the per-group table together with the H–L statistic.

The Hosmer–Lemeshow table also carries ECE and ICI, so the test is never read alone. Every calibration result has an interval from the same bootstrap replicates as the other axes, where the event count allows one.

A calibration gap inherits the event floor of its subgroups

- `max_intercept_gap`: the largest intercept difference between subgroups.
- `max_slope_gap`: the same for the slope.

A gap has no events of its own. If one of its subgroups is below the event floor, the gap has no interval.

In the worked example, the largest reserve gap, 1.62, involves the non-frail subgroup with 6 events. It has no interval. The frail minus pre-frail gap is 0.49 (0.28 to 0.71).

Common mistake. Quoting the largest gap. It usually comes from the smallest subgroup.

The gap is stored under its own name, max_intercept_gap, and not as a calibration-intercept row. That way its verdict can differ from each subgroup's verdict. The estimable contrast in the worked example is frail against pre-frail. It is the headline result of the paper: the model overestimates more as physiological reserve increases. The next slide shows the main limitation of that result.

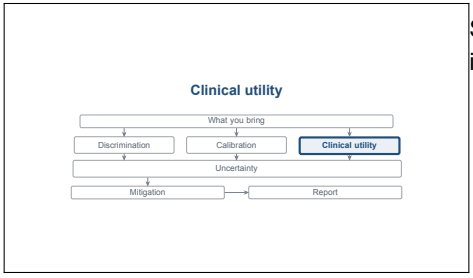
The outcome was recorded more completely in frail patients

Among patients with an infection code in 2018–2022, the share recorded as an infection after surgery:

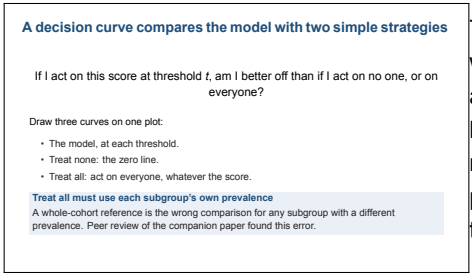
Non-frail	15.0% (7.1 to 29.1)
Pre-frail	24.2% (20.2 to 28.6)
Frail	33.5% (30.9 to 36.3)

The present-on-admission field was filled in for 92.5% to 98.0% of patients in each subgroup. Recording may explain part of the reserve gradient, so the gradient describes the model together with the recorded outcome.

If infections in pre-frail and non-frail patients are recorded less often, the observed rate in those subgroups is too low. Then the model looks more overestimating there than it is. The field was filled in almost everywhere, so missing fields do not explain the difference. It may partly reflect real differences in when infections start. The paper therefore treats the result as a property of the model and the coded outcome together. It does not attribute it to coding behaviour.



Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.



The decision curve answers the question a clinician would ask. It does not ask whether the model is accurate. It asks whether using it at a given threshold beats the two simplest strategies. In isitfair, the treat-all reference is computed for each subgroup from its own prevalence. The package wraps the dcurves library for the decision curve itself.

Net benefit counts correct flags and charges for false alarms

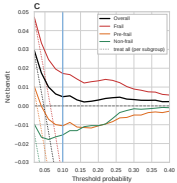
$$\mathrm{NB}_g(t) = \frac{TP_g(t)}{n_g} - \frac{FP_g(t)}{n_g} \cdot \frac{t}{1-t}$$

In words. Count the patients correctly flagged. Subtract the false alarms, each weighted by what your threshold says a false alarm costs.

- Units: net true positives per patient. Multiply by 1,000 to report.
- $\mathrm{NB}_g(t) > 0$: better than acting on no one.
- Above the treat-all curve: better than acting on everyone.
- $\mathrm{NB}_g(t) < 0$: acting on the score is worse than ignoring it.

Net benefit is the axis that turns a calibration finding into a decision about use. Clinicians find ”per 1,000 patients” easier to read than a value per patient. A negative net benefit in a subgroup means that, in that subgroup, the false alarms cost more than the infections found are worth, at the chosen threshold.

In the worked example, net benefit differed by reserve



Per 1,000 operations at $t = 0.10$, 2014–2022:

Frail	+17.21
Pre-frail	−10.59

Pre-frail in 2018–2022: +3.48 (−5.80 to +13.55).

Non-frail has 6 events, below the 50-event floor for net benefit.

The vertical line is the 10% threshold. Each subgroup's curve should be read against its own treat-all line. In frail patients the model is above both references across the plausible range. In pre-frail patients, in the full cohort, its point estimate is negative at every threshold from 5% to 20%, and its interval excludes zero from 7.5%. In the 2018–2022 window the pre-frail interval includes zero. So the negative value is a finding of the source cohort and should not be read as a stable subgroup effect.

The threshold sets the exchange rate between two harms

Acting at $t = 0.10$ means

$$\frac{t}{1-t} = \frac{0.10}{0.90} = \frac{1}{9},$$

finding one infection is worth nine unnecessary interventions.

This follows from choosing 0.10

It is not evidence that clinicians accept this exchange. Say which one you have.

Treat the threshold as a reference point, and test it:

- Report net benefit over a plausible range, such as 0.05 to 0.20.
- Draw the curve over a wider range, such as 0.02 to 0.40.

In the worked example, 0.10 was prespecified as a reference threshold for closer monitoring. It was not derived from evidence about clinicians' preferences. Showing results across a range of thresholds protects you: if the sign of a finding changes at 0.075, a table at 0.10 alone hides that.

Standardized net benefit allows comparison between subgroups

Net benefit grows with prevalence. Divide by the subgroup's own prevalence:

$$\mathrm{sNB}_g(t) = \frac{\mathrm{NB}_g(t)}{\hat{\pi}_g}$$

In words. The share of the benefit a perfect model could give in this subgroup. sNB = 1 matches a perfect model. sNB ≤ 0 is no better than acting on no one.

In the worked example at $t = 0.10$, in 2014–2022: frail 0.26, pre-frail −0.38.

Common mistake. Comparing raw net benefit between subgroups. A subgroup with low prevalence has less benefit to gain.

In frail patients the model captured about a quarter of the benefit a perfect model would capture. In pre-frail patients it lost benefit. Like net benefit itself, these values depend on the recorded prevalence, so they belong to the 2014–2022 window.

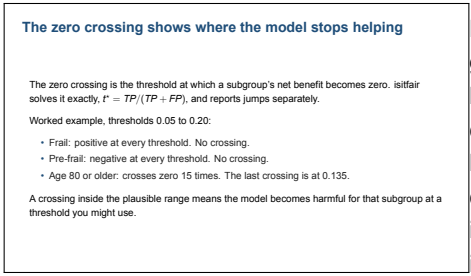
The utility gap is the largest disagreement over the range

$$\Delta \mathrm{NB}_{max} = \max_{t \in [0.02,\ 0.40]} \max_{g,g'} \left| \mathrm{NB}_g(t) - \mathrm{NB}_{g'}(t) \right|$$

In words. The largest difference in net benefit between any two subgroups, at any threshold you might plausibly use.

Its interval comes from the same bootstrap replicates as the subgroup values.

This was the primary clinical-utility gap in the paper. Because it is a maximum over thresholds and pairs, it is computed inside every bootstrap replicate, from the same resampled patients. That keeps the dependence between the gap and the subgroup values it is built from.



Between two points where a patient enters the flagged group, the flagged group does not change, so net benefit is a smooth function of t and the zero has a closed form. At the points where a patient enters, net benefit jumps. Many small jumps produce many crossings, as in the oldest age group. The last crossing is the useful summary: above 0.135, acting on the score in patients aged 80 or older is worse than doing nothing.

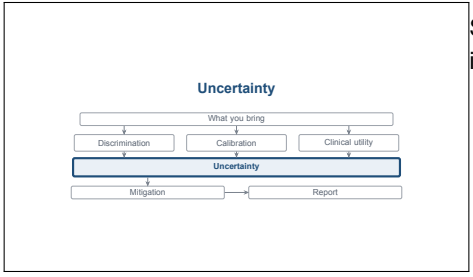
Run clinical utility

```
dca = audit.decision_curve(
    thresholds=[0.05, 0.075, 0.10, 0.125, 0.15, 0.175, 0.20],
    zero_crossing=True,
)
# net_benefit and standardized_net_benefit, each with a CI,
# treat_all_nb, treat_none_nb, zero_crossing, GAP rows

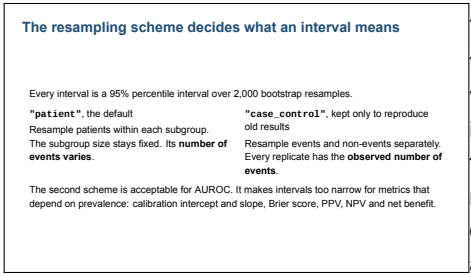
curve = audit.decision_curve()    # default: 20 points, 0.02 to 0.40
```

`treat_all_nb` is already computed per subgroup. Values are per patient: multiply by 1,000 to report.

Report utility over a range of thresholds. The default grid of the plain call is meant for drawing the curve. The explicit list is the table you report.



Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.



The question is what varies between replicates. With the patient scheme, the number of events in a subgroup varies as it would in a new sample, so the interval includes that variation. The case-control scheme holds the number of events fixed. It answers how uncertain a result is if exactly the observed number of events occurs. That is a narrower question than the one an audit asks. The first submission of the paper used it.

With six events, the two schemes gave very different intervals

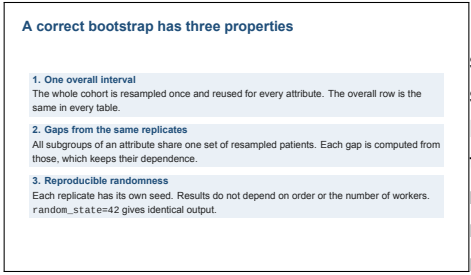
Calibration intercept of the non-frail subgroup, 6 events:

Scheme	95% interval	Width (logits)
`case_control`	−2.55 to −2.40	0.15
`patient`	withheld	1.76

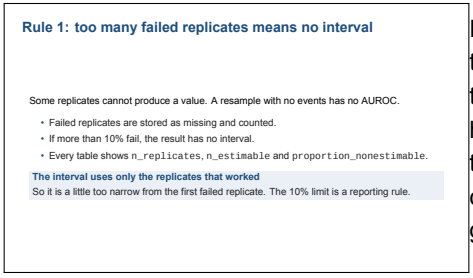
The patient interval is more than 10 times wider.

case_control can never draw a replicate without events
So its warning for sparse subgroups always reads 0%. It cannot warn you.

The patient-scheme interval is withheld because the subgroup is below the event floor. Its width is shown here only to compare the two schemes. The narrow interval is not careful; it answers a question nobody asked. Use the patient scheme. The other one exists in the package only so that the originally submitted analysis can still be reproduced.



If your table shows a different overall interval in the sex section than in the age section, the bootstrap was run separately per attribute, and a reviewer will notice. Drawing a gap from a new resample breaks its link to the subgroup values it is built from. The seed of replicate b of attribute a is derived from those two numbers, so running in parallel does not change any result.



Failed replicates are never dropped silently. Because the percentile interval is computed over the replicates that worked, it is conditional on the subgroup having had enough events. That makes it narrower than the true interval, from the first failure onward. The 10% cut-off says when we stop reporting it. It does not guarantee that intervals below 10% are exact.

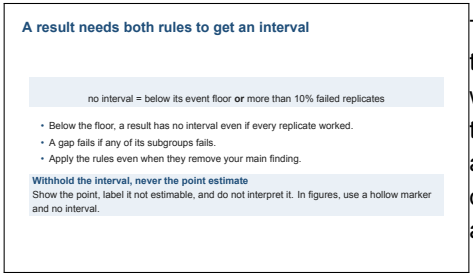
Rule 2: each metric needs a minimum number of events

Metric	Minimum events
AUROC	25
Calibration intercept	25
Calibration slope	100
Net benefit	50

In words. Below these counts the metric is mostly noise. The slope needs the most events, because it estimates a gradient.

Common mistake. Setting floors after seeing the results. Fix them before the analysis.

These floors were prespecified for the companion study. AUPRC, sensitivity, specificity and predictive values follow the AUROC floor, and the Brier score follows the calibration-intercept floor. For AUROC, the standard error is roughly 0.5 divided by the square root of the number of events, which gives about 0.1 at 25 events. You may use other floors, but write down why before the analysis. Afterwards, every incentive points toward an exception.



The floor is a fixed rule. The value of prespecifying it is that you apply it everywhere, including to results you would like to report. A point estimate is a real number that was computed. Deleting it would hide what you audited. In the paper's figures, sparse subgroups are drawn with hollow markers or dashed curves, so they are visible and clearly unsupported.

Ask the audit which results may carry an interval

```
audit = FairnessAudit(
    y, p_oof, groups, threshold=0.10,
    resample_scheme="patient", n_bootstrap=2000, random_state=42,
    nonestimable_threshold=0.10,       # rule 1
    event_floors={                     # rule 2
        "auroc": 25, "calibration_intercept": 25,
        "calibration_slope": 100, "net_benefit": 50,
    },
)
est = audit.estimability()
# metric, n_events, event_floor, below_floor,
# proportion_nonestimable, not_estimable_final, rule
est[est.not_estimable_final]   # results without an interval
```

Save this table next to your results. Later code should read it instead of re-applying the floors.

The values shown are the defaults, so you only need to pass them if you change them. The rule column says in words which rule removed the interval. If the same rule is written in five scripts, it soon becomes five different rules. Reading one saved table avoids that.

Correct for multiple testing within each family of tests

Use the Benjamini–Hochberg false discovery rate (BH-FDR) at $q = 0.05$, within each prespecified family. Sort the p-values, find the largest k with $p_{(k)} \le (k/m)\,q$, and reject the first k.

In words. The more tests in a family, the stricter each test. FDR limits the share of false findings among your findings.

Families in the worked example: the gaps of each axis, a 23-test sex by comorbidity screen, and the Table 1 comparisons. Subgroup estimates are descriptions, so they are not corrected.

Common mistake. Putting every p-value of a paper into one family.

FDR is less strict than controlling the chance of any single false finding, and that is the right trade for a screen. One family for the whole paper would make every test so strict that you lose the findings you planned, to protect against tests you never meant to run. The function fdr_correct wraps the statsmodels implementation.

An interaction test compares an effect between groups

A gap shows what one threshold does. To ask whether a risk factor *X* acts differently by attribute *A*, compare two logistic models:

$$M_0: \ \text{logit}\,\Pr(Y{=}1) = \beta_0 + \beta_X X + \beta_A A + \beta_C^{\top} C$$

$$M_1: \ \text{logit}\,\Pr(Y{=}1) = \beta_0 + \beta_X X + \beta_A A + \beta_{XA} XA + \beta_C^{\top} C$$

$$\text{LR} = 2\left(\ell(M_1) - \ell(M_0)\right) \sim \chi^2_{df}$$

In words. Does the extra term improve the fit enough to justify it? The test is for $\beta_{XA} = 0$.

C are the adjustment variables and ell is the log-likelihood. The degrees of freedom equal the number of interaction terms. A within-group odds ratio, for example for diabetes in women, is an association inside one group. It does not test whether the association differs between groups. Only the interaction term does that.

Report the interaction p-value with its effect size

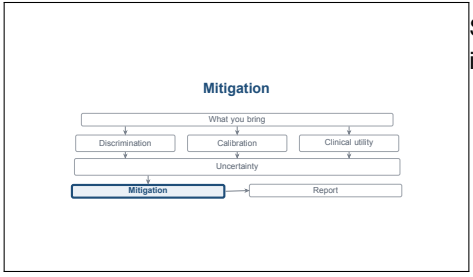
```
from isitfair import interaction_test

res = interaction_test(
    df, outcome="y", sensitive="sex", covariate="comorbidity",
    adjust_for=("age_years", "asa_class"),   # numeric columns
)
res.p_value, res.delta_r2
```

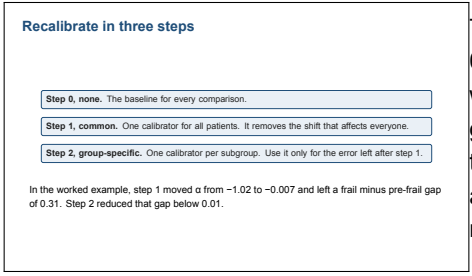
`delta_r2` is the increase in Nagelkerke R^2 from the interaction term.

In the worked example, sex by physiological reserve gave p = 0.018 and $\Delta R^2 = 0.0008$. The effect is significant and very small.

With more than ten thousand operations, a tiny effect is still significant, so always report the effect size next to the p-value. The adjustment columns enter the model as linear terms, so they must be numeric. Categorical adjustment variables need to be encoded first.



Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.



The common step had an interval of minus 0.116 to 0.091 for the overall intercept, and the remaining gap was 0.31 with interval 0.07 to 0.55. If you skip step 1, a group-specific correction will also appear to work. But then a problem that affects everyone has been attributed to a subgroup, and the model uses group membership without need.

Isotonic recalibration changes risk values and keeps their order

Per subgroup g, fit a step function ι that never decreases:

$$\iota_g = \arg\min_{\iota} \sum_{i \in I_g^{val}} \left(\iota(p_i) - Y_i\right)^2, \qquad p_i^{iso} = \iota_g(p_i)$$

In words. Fit a staircase that only goes up. The order of the patients stays the same. Only the values change.

- AUROC is unchanged, except for ties in very small subgroups.
- It can fix curve shapes that a logistic recalibration cannot.

Common mistake. Trusting it in small subgroups. Check the slope after recalibration together with the event count.

The result is still a probability, so every later step works on it unchanged. The package uses scikit-learn's isotonic regression. Platt and beta calibration are also available through the method argument. Isotonic regression overfits subgroups with few events, as the next slides show.

Fit the recalibration with cross-fitting

```
ladder = audit.recalibration_ladder(
    by="reserve",
    method="isotonic",
    folds=model_fold_ids,     # the model's own CV folds
)
# one row per step ("none", "common", "group_specific")
# and subgroup: intercept, slope, ECE, ICI, Brier,
# net benefit at t, and the intercept gap
```

Each calibrator is fitted on four folds and applied to the fifth. Every patient gets a recalibrated risk from a calibrator that did not see them.

Common mistake. Drawing new folds when the model was cross-validated. Pass the model's own fold IDs.

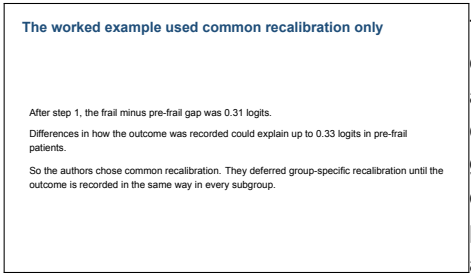
A calibrator fitted and evaluated on the same patients looks excellent and means nothing. With cross-fitting, the before and after comparison uses the whole cohort. Reusing the model's folds matters: with new folds, a calibrator can be fitted on patients that the model was tested on, and the leakage is different from what you documented. Without folds, the package draws cv folds with the audit's random state.

Group-specific recalibration collapsed with six events

- Non-frail: 6 events, below every event floor.
- Its isotonic map gave the 1,022 patients only 17 different risk values.
- Slope after recalibration: **0.12** (against 0.93 overall).

The average looks corrected because the map is almost constant.

Before recalibration, the non-frail patients had 64 different risk values. A slope of 0.12 means the recalibrated risks are spread far too wide relative to what they predict. This is a diagnostic result, and we do not interpret it as evidence about non-frail patients. It is a reason not to fit a separate calibrator in a subgroup with six events.



The 0.33 logits is an upper bound. It assumes that every unrecorded infection in the pre-frail subgroup was a real infection after surgery. The remaining gap after common recalibration is about the same size. A group-specific calibrator would then encode a data-recording difference as if it were a property of pre-frail patients. Recorded sex should also not be used as a stand-in mitigation axis.

Wasserstein post-processing aligns score distributions

$$Q_{\text{bary}}(u) = \sum_g w_g F_g^{-1}(u), \qquad \hat{p}^W = Q_{\text{bary}}(F_g(p))$$

In words. Line up each subgroup's scores by percentile and average them. The order inside each subgroup is kept, so subgroup AUROC does not change.

```
from isitfair import WassersteinPostprocessing

w = audit.mitigate(
    method=WassersteinPostprocessing(constraint="demographic_parity"),
    calibration_data=(y_cal, p_cal, groups_cal),
)
```

Fit it on a separate calibration set. The audited patients stay unseen.

F g is the distribution of scores in subgroup g, and w g its weight. The barycenter is the average distribution. After the transform, every subgroup has the same score distribution, so the same share of patients is flagged at any threshold. That is demographic parity.

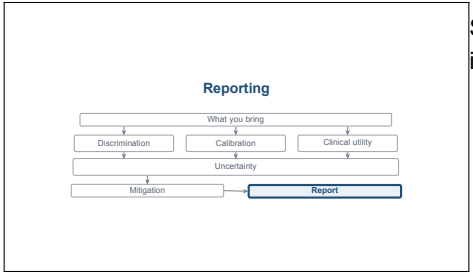
Aligned score distributions make calibration worse

When prevalence differs between subgroups, a score cannot satisfy both calibration and equalized odds.

- Use Wasserstein post-processing as a comparison for parity.
- Expect calibration to get worse.
- If the problem is miscalibration, use recalibration instead.

In the worked example, it was a comparison on the procedure-cluster axis.

Pleiss and colleagues proved this as a theorem, so more data cannot remove it. A transform that gives every subgroup the same score distribution moves risks away from the observed rates in subgroups with higher or lower prevalence. Isotonic recalibration matches the problem found in the worked example. Neither method can repair weak ranking, which needs better data or a better model.



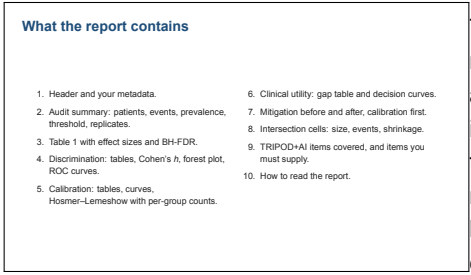
Section page. Point to the marked box on the map: this is the step of the audit the next slides cover.

Write the report

```
audit.report(
    out="audit_report.html",
    title="My model: subgroup audit",
    metadata={"Model": "XGBoost v3", "Threshold": "0.10",
              "Predictions": "out-of-fold, 5-fold CV"},
    mitigation_audits=mitigated,    # optional before and after section
    table_one_data=cohort_df,       # optional cohort table
    pdf=True,                       # needs the isitfair[pdf] extra
)
```

One HTML file with all figures inside and no JavaScript. Figures are also saved as 300-DPI PNG files. With a fixed `random_state`, the report is identical on every run.

The figures folder is written next to the report, ready for a manuscript. The PDF copy needs WeasyPrint, installed with the pdf extra. Without it, the HTML report is still written and only the PDF call fails, with a message that names the extra. Because the report is identical on every run, you can regenerate it later from the saved audit.



TRIPOD+AI is the reporting guideline for prediction models built with regression or machine learning. The appendix marks the items the package covers and the items only you can supply, such as the data source and the intended use. It is a checklist. It does not certify the model. In the mitigation section, discrimination is labelled as a check, because recalibration should not change ranking.

Small intersection cells are pulled toward their parent subgroup

Cells with fewer than 50 events have their rate shrunk toward the parent subgroup's rate:

$$\tilde{r}_c = (1 - w_c)\, r_c + w_c\, r_0, \qquad w_c = \frac{\tau}{n_c^2 + \tau}$$

In words. The fewer events a cell has, the more its rate is pulled toward the parent. $w_c = 0$ means no shrinkage, $w_c = 1$ full pooling.

- Only rates are shrunk. AUROC and AUPRC are not.
- `cell_diagnostics()` reports w_c for every cell.

Common mistake. Reading a shrunk cell as an estimate of that cell alone.

Tau is estimated from the variation between cells. The cut-off of 50 events is the default, set with intersectional_min_events. Shrunk rows have shrunk set to true, and their discrimination intervals are withheld. In the worked example, the intersections mostly repeated the age and reserve findings and added no new pattern.

The whole audit in code, part 1

```
from isitfair import FairnessAudit

audit = FairnessAudit(
    y_true=y, y_proba=p_oof,
    sensitive_features={"sex": sex, "age_group": age,
                        "reserve": reserve,
                        ("sex", "reserve"): None},
    threshold=0.10, n_bootstrap=2000, random_state=42,
)

results = audit.run_all()
# discrimination, roc_curves, calibration, calibration_curves,
# hosmer_lemeshow, decision_curve, estimability
```

run_all computes every axis once. Each result is stored, so calling a single axis afterwards costs nothing. The estimability table is part of run_all, because the results and the rules about them belong together.

The whole audit in code, part 2

```
for name, table in results.items():
    table.to_csv(f"audit_{name}.csv", index=False)

audit.report(out="audit_report.html",
             title="Subgroup audit")
```

Later scripts read `audit_estimability.csv`. They do not re-apply the event floors.

Expect about a minute at 200 replicates and 10 to 15 minutes at 2,000 for a cohort of about 2,000 patients and three attributes, on a laptop. Start with 200 replicates to check that everything runs. Then use 2,000 for the results you report.

compare_periods puts two audits side by side

```
from isitfair import compare_periods

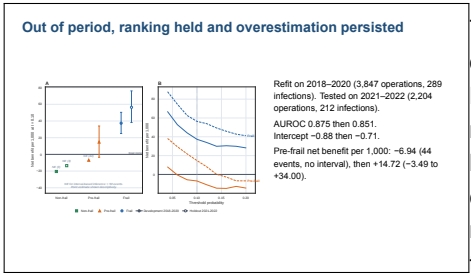
dev  = FairnessAudit(y_dev,  p_dev,  groups_dev,  threshold=0.10)
test = FairnessAudit(y_test, p_test, groups_test, threshold=0.10)

by_period = compare_periods({"2018-2020": dev, "2021-2022": test})
```

isitfair does not refit your model. Your own pipeline refits on the first period, with all preprocessing, feature selection and resampling fitted on that period only.

Common mistake. Reading a difference between periods as model drift. Recording practices also change over time.

The package evaluates predictions. Training is outside its scope on purpose. Before you attribute a change to the model, compare case mix and how the outcome was recorded in each period. The next slide shows this in the worked example.



This experiment retrains the model on years where the outcome is recorded, so it avoids the pre-2018 problem. Ranking held, the model still overestimated risk, and the reserve gradient persisted at a smaller size. Frail net benefit was positive in both periods. Recording also changed: 28.8% of eligible infected patients were recorded as cases in development, and 34.0% in the test period. So part of the difference between periods comes from recording. This is an out-of-period test. It is not an external validation of the published model.

Fix these decisions before you look at any result

1. The subgroup axes, and which of them are case-mix axes.
2. The primary axis for your use, and so what counts as a finding.
3. The threshold, and the range you will test around it.
4. The event floors, with a written reason.
5. The test families for FDR correction.
6. That predictions are out-of-fold, and that any recalibration will be cross-fitted.

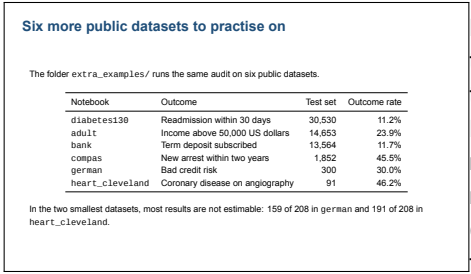
Each of these is easy to decide in advance and hard to defend if you decide it after seeing the results. Put them in a protocol or a pre-registration. The companion paper prespecified all six.

What to write in a finished audit

- Subgroup values on all three axes, with intervals where the data allow, and “not estimable” where they do not.
- Gaps with effect sizes, read against case mix and prevalence.
- The primary criterion, and what it means for this score at this threshold.
- The mitigation steps you tried, in order, and the ones you rejected, with reasons.
- What is still unresolved, and which data would resolve it.

Do not write that the model is fair. None of these numbers can show that.

A reader should be able to see what the model does in each subgroup, how sure we are, and what the authors decided to do about it. In the worked example, the unresolved part is the outcome recording. The paper says that prospective validation with comparable recording in every subgroup is needed before clinical use.



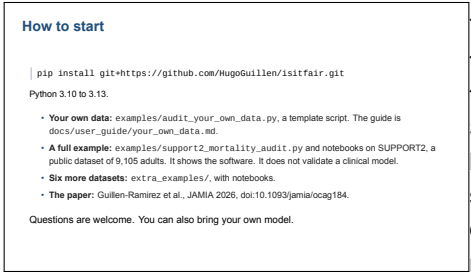
Six more public datasets to practise on

The folder `extra_examples/` runs the same audit on six public datasets.

Notebook	Outcome	Test set	Outcome rate
`diabetes130`	Readmission within 30 days	30,530	11.2%
`adult`	Income above 50,000 US dollars	14,653	23.9%
`bank`	Term deposit subscribed	13,564	11.7%
`compas`	New arrest within two years	1,852	45.5%
`german`	Bad credit risk	300	30.0%
`heart_cleveland`	Coronary disease on angiography	91	46.2%

In the two smallest datasets, most results are not estimable: 159 of 208 in `german` and 191 of 208 in `heart_cleveland`.

Each notebook follows the same twelve steps as this talk: load the predictions, split off a calibration half, build the audit, the three axes, estimability, Hosmer-Lemeshow, the intersection cells, the recalibration ladder, mitigation and the report. The model is a plain logistic regression, refitted by `datasets.py`, which downloads the data from UCI and from ProPublica. The data are not shipped with the package, so check each source's terms of use. Only two of the six are clinical: diabetes130 and heart_cleveland. The others are standard fairness benchmarks, useful to practise the method but far from the clinical setting the package is built for. Two notebooks teach something extra. In diabetes130, 676 people have no race recorded, and isitfair refuses to run until they are recoded as an explicit category. The two small datasets show the estimability rule at work: with 91 or 300 people, most subgroup results get a point estimate and no interval. The runs use 50 bootstrap replicates to stay fast, so use 2,000 for any result you report.



The template script needs only a CSV with the outcome, the predicted probability and the subgroup columns. Set the column names at the top and run it. The SUPPORT2 notebooks run offline, because the data are bundled. The tutorial notebook follows these slides section by section. The extra examples download their data, so they need the examples extra: in a clone of the repository, run `pip install -e ".[examples]"`.